

WIRESHARK

To Listen the Telephone voice in Wireshark

- The in the main> Telephony> VoIP Calls: List is there
 - The Protocol in MiNT
 - Right Click> Decode > RTP then Go to Telephony “Top Menu”> RTP> Analyze > Play

NOTE: Frame is layer2 data i.e Data link (Eth0 or Wireless): Packet at layer 3 (Network) and Segment in Layer 4(Transport) in TCP Model top is Application

i.e in Layer 4: you get the browser used

- To get the image, in layer 4> scroll down to Portable Network Graphics > Right Click> Export to i.e format is .bin, then later save to png and open in your folder

DNS and DHCP Protocols Analysis

- DNS is port 53, remember in
- layer 4 is where to get the Protocol and the browser Agent
- Layer 3: Ethernet here we see the MAC address for source and destination

How to Search Packet

- Tcp.port eq 25 or icmp
- Ip.src==192.168.1.1
- Protocol.property

TCPDUMP

0. How to capture:

\$ tcpdump -i eth0 -v src/dst <ip> or use wireshark

1. How many unique IP addresses are communicating in this PCAP file?

To list them:

```
$ tcpdump -nnr capture.pcap | awk '{print $3,$5}' | tr -d ':' | tr ' >' '\n' | grep -E '^[0-9]+\.'
```

To count them:

```
$ tcpdump -nnr capture.pcap | awk '{print $3,$5}' | tr -d ':' | tr ' >' '\n' | grep -E '^[0-9]+\.'
```

In Wireshark: Statistics> Endpoints: IPv4, IPv6

2. Which IP address generated the most traffic, and how did you determine it?

```
$ tcpdump -nnr capture.pcap | awk '{print $3}' | cut -d. -f1-4 | sort | uniq -c | sort -nr | head
```

In Wireshark: Statistics> Endpoints: IPv4, IPv6 > Sort by packet

3. What are the top destination ports being accessed in this capture?

```
$ tcpdump -nnr capture.pcap | awk '{print $5}' | grep -oE '\.[0-9]+:' | tr -d ':' | sort | uniq -c | sort -nr | head
```

4. Identify all HTTP requests in the PCAP and determine which hostnames (domains) were visited?

i.e., to get the hosts:

```
$ tcpdump -A -r capture.pcap | grep "HTTP"
```

In Wireshark: filter: http.host/ http.request

5. To determine the Browser Agent or OS:

```
$ tcpdump -A -r capture.pcap | grep "User-Agent:"
```

In Wireshark: chini pale in application layer segment, unapata kila kitu

In Wireshark: filter: http.user_agent

6. To get a downloaded file:

```
$ tcpdump -A -nnr capture.pcap 'tcp port 80' | grep -i "filename="
```

7. Is there any evidence of credential transmission (usernames, passwords, tokens, API keys) in clear text?

```
$ tcpdump -A -r capture.pcap | egrep -i  
"user|username|login|pass|password|token|apikey|api_key|authorization"
```

NOTE: -r reads the packet instead of reading from live

NOTE: -A set the output to ASCII "human readable format"

SMTP: tcpdump -nn -A -r capture.pcap port 25 | egrep mail to, mail from, subject

```
echo "SFRce2Jhc2U2NF9mbGFnfQ==" | base64 -d
```

NOTE: The files in Wireshark.

filter: tcp.stream eq 2 > right click > follow > change format to raw > save > find the file type by running the command file <filename> then convert to respective extension and proceed

OWASP TOP 10

A01:2025 – Broken Access Control

First check if you can get folder or url

\$ gobuster dir -u http://127.0.0.1:42001 -w /usr/share/wordlists/dirb/common.txt

```
(root@kali)-[/home/kali/Downloads]
# strings file.txt
*UJS

(root@kali)-[/home/kali/Downloads]
# gobuster dir -u http://127.0.0.1:42001 -w /usr/share/wordlists/dirb/common.txt

Gobuster v3.8.2
by OJ Reeves (@TheColonial) & Christian Mehlmauer (@firefart)

[+] Url: http://127.0.0.1:42001
[+] Method: GET
[+] Threads: 10
[+] Wordlist: /usr/share/wordlists/dirb/common.txt
[+] Negative Status codes: 404
[+] User Agent: gobuster/3.8.2
[+] Timeout: 10s

Starting gobuster in directory enumeration mode

config (Status: 301) [Size: 169] [→ http://127.0.0.1:42001/config/]
database (Status: 301) [Size: 169] [→ http://127.0.0.1:42001/database/]
docs (Status: 301) [Size: 169] [→ http://127.0.0.1:42001/docs/]
external (Status: 301) [Size: 169] [→ http://127.0.0.1:42001/external/]
favicon.ico (Status: 301) [Size: 1406]
```

What it is

Users can access resources or perform actions they should not be allowed to.

Realistic Vulnerability Scenarios

1. Normal user accesses admin page directly:

```
/admin
/admin/users
```

2. IDOR (Insecure Direct Object Reference):

```
/api/user?id=1001
/api/user?id=1002
```

3. User role changed via parameter tampering:

```
role=user → role=admin
```

Tools

- curl
- Burp Suite
- ffuf

- Browser DevTools

Techniques & Commands (Linux)

◆ Test unauthorized access

```
curl -i http://target/app/admin
```

◆ IDOR testing

```
curl -H "Authorization: Bearer USER_TOKEN" \  
http://target/api/orders/1001
```

Change 1001 → 1002

◆ Parameter tampering

```
curl -X POST http://target/update-role \  
-d "user_id=10&role=admin"
```

Interview Explanation

“Broken Access Control occurs when authorization checks are missing or enforced only on the client side. I test this by accessing restricted endpoints directly and modifying object identifiers.”

Perfect. This is a **very strong interview-focused approach**.

We’ll do **A01–A04** in a **practical, attacker + defender mindset**, with:

-  **Realistic vulnerability scenarios**
-  **Tools**
-  **Linux commands**
-  **What to say in an interview**

I’ll assume you’re practicing on **DVWA / test apps / lab environments**.

A01:2025 – Broken Access Control

What it is

Users can **access resources or perform actions they should not be allowed to.**

Realistic Vulnerability Scenarios

1. Normal user accesses admin page directly:
 2. /admin
 3. /admin/users
 4. IDOR (Insecure Direct Object Reference):
 5. /api/user?id=1001
 6. /api/user?id=1002
 7. User role changed via parameter tampering:
 8. role=user → role=admin
-

Tools

- curl
 - Burp Suite
 - ffuf
 - Browser DevTools
-

Techniques & Commands (Linux)

Test unauthorized access

```
curl -i http://target/app/admin
```

IDOR testing

```
curl -H "Authorization: Bearer USER_TOKEN" \  
http://target/api/orders/1001
```

Change 1001 → 1002

Parameter tampering

```
curl -X POST http://target/update-role \  
-d "user_id=10&role=admin"
```

Interview Explanation

“Broken Access Control occurs when authorization checks are missing or enforced only on the client side. I test this by accessing restricted endpoints directly and modifying object identifiers.”

A02:2025 – Security Misconfiguration

What it is

The application or server is **incorrectly configured**, exposing sensitive data or functionality.

Realistic Vulnerability Scenarios

1. Debug mode enabled
 2. Default credentials
 3. Directory listing enabled
 4. Exposed admin panels
 5. Stack traces shown to users
-

Tools

- nmap
 - nikto
 - curl
 - whatweb
-

Techniques & Commands

Scan for common misconfigurations

```
nikto -h http://target
```

Check open ports & services

```
nmap -sV -Pn target
```

◆ Test directory listing

```
curl http://target/uploads/
```

◆ Check headers

```
curl -I http://target
```

Interview Explanation

“Security misconfiguration happens when unnecessary features, services, or debug settings are exposed. I use tools like Nikto and manual inspection to identify these.”

A03:2025 – Software Supply Chain Failures

What it is

Risk introduced by **third-party libraries, dependencies, or CI/CD pipelines**.

Realistic Vulnerability Scenarios

1. Vulnerable NPM / Python package
 2. Outdated OpenSSL library
 3. Malicious dependency injection
 4. No integrity verification for updates
-

Tools

- npm audit
 - pip-audit
 - trivy
 - syft
-

Techniques & Commands

◆ Node.js dependency scan


```
npm audit
```

◆ Python dependency scan

```
pip-audit
```

◆ Container image scan

```
trivy image myapp:latest
```

◆ Generate SBOM

```
syft dir:.
```

Interview Explanation

“Supply chain failures occur when vulnerable or malicious third-party components are used. I detect these by auditing dependencies and generating SBOMs.”

A04:2025 – Cryptographic Failures

What it is

Sensitive data is **not properly encrypted** or crypto is used incorrectly.

Realistic Vulnerability Scenarios

1. HTTP instead of HTTPS
 2. Weak hashing (MD5, SHA1)
 3. Hardcoded encryption keys
 4. Sensitive data in plaintext
 5. Broken TLS configuration
-

Tools

- openssl
- testssl.sh
- sslscan

- `grep`

Techniques & Commands

◆ Check HTTPS / TLS

```
openssl s_client -connect target:443
```

◆ Scan TLS weaknesses

```
testssl.sh target
```

◆ Search for weak hashes

```
grep -R "md5\|sha1" /var/www/
```

◆ Capture plaintext credentials (lab only!)

```
tcpdump -A -i eth0 port 80
```

Interview Explanation

“Cryptographic failures occur when data is not encrypted properly or weak algorithms are used. I assess this by inspecting TLS configurations and checking how sensitive data is stored and transmitted.”

Summary Table (Interview Gold)

OWASP ID	Core Risk	Common Tool
A01	Authorization bypass	curl, Burp
A02	Bad configs	nikto, nmap
A03	Vulnerable dependencies	npm audit, trivy
A04	Weak crypto	openssl, testssl

Great — continuing exactly in the same **hands-on, interview-ready** style.

Below are **OWASP Top 10:2025 – A05 to A08**, with **real scenarios, attacker techniques, Linux tools, commands**, and **what to say in interviews**.

I'll assume **lab / DVWA / test apps only**.

OWASP Top 10:2025 — Part 2 (A05–A08)

(Reference framework: OWASP)

A05:2025 – Injection

What it is

Untrusted data is sent to an interpreter (SQL, OS, LDAP, NoSQL), allowing **command or query execution**.

Realistic Vulnerability Scenarios

1. SQL Injection

```
' OR '1'='1
```

2. Command Injection

```
; ls -la
```

3. NoSQL Injection

```
{"$ne": null}
```

Tools

- sqlmap
- curl
- Burp Suite
- wfuzz

Techniques & Linux Commands

◆ Test SQL injection manually

```
curl "http://target/login.php?user=admin'--&pass=test"
```

◆ Automated SQLi testing

```
sqlmap -u "http://target/item.php?id=1" --dbs
```

◆ Command injection test

```
curl "http://target/ping.php?ip=127.0.0.1;whoami"
```

Interview Explanation

“Injection flaws occur when user input is concatenated into queries or system commands. I test this by injecting payloads and verifying execution through application responses.”

A06:2025 – Insecure Design

What it is

Security **was never properly designed**, even if implementation is correct.

This is **architectural**, not just coding.

Realistic Vulnerability Scenarios

1. No rate limiting on login
 2. Password reset without verification
 3. Business logic abuse:
 - Negative prices
 - Reusing coupons
 4. Trusting client-side validation
-

Tools

- Browser DevTools
 - curl
 - Burp Repeater
 - Threat modeling (STRIDE)
-

Techniques & Linux Commands

◆ Business logic abuse

```
curl -X POST http://target/checkout \  
-d "item=phone&price=-100"
```

◆ Password reset abuse

```
curl -X POST http://target/reset-password \  
-d "email=victim@company.com"
```

◆ Rate-limit testing

```
for i in {1..50}; do  
  curl -X POST http://target/login \  
  -d "user=admin&pass=wrong"  
done
```

Interview Explanation

“Insecure design means security controls were never considered during architecture. Even correct code can be insecure if business rules are flawed.”

A07:2025 – Authentication Failures

What it is

Weak authentication mechanisms allow attackers to **impersonate users**.

Realistic Vulnerability Scenarios

1. No account logout
 2. Weak passwords
 3. Session IDs in URLs
 4. Token reuse
 5. Missing MFA
-

Tools

- hydra
 - ffuf
 - Burp Intruder
 - crunch
-

Techniques & Linux Commands

◆ Brute-force login

```
hydra -l admin -P passwords.txt target http-post-form \
"/login.php:user=^USER^&pass=^PASS^:Invalid"
```

◆ Session fixation test

```
curl -i http://target/dashboard?sessionid=12345
```

◆ Password spraying

```
hydra -L users.txt -p Welcome@123 target ssh
```

Interview Explanation

“Authentication failures happen when identity verification is weak. I test for brute force, session handling issues, and missing protections like MFA.”

A08:2025 – Software or Data Integrity Failures

What it is

The application **trusts unverified data or code**, allowing tampering.

💣 Realistic Vulnerability Scenarios

1. Unsigned software updates
2. Insecure deserialization
3. Tampered cookies / JWTs
4. CI/CD pipeline compromise

🧰 Tools

- `jwt-tool`
- `hashcat`
- `openssl`
- `ysoserial`

🔧 Techniques & Linux Commands

◆ Decode & tamper JWT

```
jwt-tool eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9
```

◆ JWT none-alg attack (lab only)

```
jwt-tool token.txt -X a
```

◆ Verify file integrity

```
sha256sum app.tar.gz
```

◆ Insecure deserialization test

```
java -jar ysoserial.jar CommonsCollections1 "id"
```

🧑 Interview Explanation

“Integrity failures occur when applications trust data without validation. I test this by tampering with tokens, serialized objects, and update mechanisms.”

✓ Quick Interview Summary Table

OWASP ID	Key Risk	Typical Attack
A05	Injection	SQL / OS injection
A06	Insecure design	Business logic abuse
A07	Auth failures	Brute force, session hijack
A08	Integrity failures	JWT tampering, deserialization

Perfect — let's **finish strong**.

These last two are **very important for SOC / government / blue-team interviews**, and many candidates explain them poorly. You'll stand out if you explain them clearly like this.

OWASP Top 10:2025 — Final Part (A09–A10)

(Framework reference: OWASP)

A09:2025 – Security Logging & Alerting Failures

What it is

The system **does not log, monitor, or alert** on security-relevant events, allowing attacks to go **undetected**.

Realistic Vulnerability Scenarios

1. Failed logins not logged
2. No alerts on brute-force attempts
3. Logs missing:
 - IP address
 - User ID
 - Timestamp
4. Logs stored locally and easily deleted
5. No correlation between systems

Tools (Attacker & Defender Perspective)

- curl
 - hydra
 - fail2ban
 - journalctl
 - SIEM (Splunk / ELK – conceptually)
-

Techniques & Linux Commands

◆ Simulate brute-force attack

```
hydra -l admin -P passwords.txt target http-post-form \  
"/login:user=^USER^&pass=^PASS^:Invalid"
```

◆ Check if logs capture the attack

```
grep "login failed" /var/log/auth.log
```

◆ View application logs

```
tail -f /var/log/apache2/access.log
```

◆ Check centralized logging (systemd)

```
journalctl -xe
```

What You Say in Interviews

“Security logging failures occur when attacks happen but are not detected. I test this by performing suspicious actions and verifying whether they are logged, timestamped, and trigger alerts.”

Bonus points:

Mention **log integrity & retention**.

A10:2025 – Mishandling of Exceptional Conditions

What it is

The application **does not handle errors, crashes, or unexpected input safely**, leading to:

- Information disclosure
 - Denial of service
 - Security bypass
-

Realistic Vulnerability Scenarios

1. Full stack traces shown to users
 2. Application crash on malformed input
 3. Division by zero errors
 4. File upload errors revealing paths
 5. API crashes with sensitive data in error responses
-

Tools

- curl
 - Burp Repeater
 - Fuzzers (ffuf)
 - Browser DevTools
-

Techniques & Linux Commands

Trigger application errors

```
curl "http://target/product?id=999999999999"
```

Type confusion

```
curl "http://target/api/user?id[]=1"
```

Null value handling

```
curl -X POST http://target/api/checkout \  
-d "price="
```

◆ Fuzz error handling

```
ffuf -u http://target/api?input=FUZZ \  
-w /usr/share/wordlists/dirb/common.txt
```

🧠 What You Say in Interviews

“Exceptional condition failures happen when applications don’t safely handle errors. I test this by sending malformed or unexpected inputs and observing whether sensitive information is exposed or the application crashes.”

📄 Final Interview Cheat Sheet (Very Powerful)

OWASP

What Interviewers Want to Hear

- A09 Attacks must be logged, monitored, and alerted
- A10 Errors should be safely handled without leaking info
-

🚩 Final Strong Interview Statement (Use This)

“I approach OWASP Top 10 practically by simulating real attack scenarios in a controlled lab, validating both exploitability and detection, and mapping findings to risk and mitigation.”

✅ You Have Now Covered All OWASP Top 10:2025

You are now ready to:

- Explain **each OWASP category confidently**
 - Demonstrate **Linux commands**
 - Speak from **hands-on experience**
-

🔥 Want to Level Up Further?

I can help you next with:

1. **OWASP Top 10 → DVWA exact labs mapping**
2. **Government interview mock Q&A**

3. **Blue-team mitigation commands**
4. **Attack → Detection → Prevention flow**

Just tell me which one you want.

Below is a **complete, interview-ready Nmap enumeration cheat sheet** covering **host discovery, ports, services, OS, scripts, vuln checks, and performance tuning** — all **Linux commands**, clearly explained so you can *say what each does* in an interview.

Tool reference: Nmap

Nmap Full Enumeration Cheat Sheet (Linux)

1 Host Discovery (Who is alive?)

◆ Ping scan (ICMP + TCP probes)

```
nmap -sn 192.168.1.0/24
```

◆ Skip ping (useful if ICMP blocked)

```
nmap -Pn 192.168.1.10
```

◆ ARP scan (local network – fastest)

```
nmap -PR 192.168.1.0/24
```

2 Port Scanning (What ports are open?)

◆ Top 1000 TCP ports (default)

```
nmap 192.168.1.10
```

◆ All TCP ports

```
nmap -p- 192.168.1.10
```

◆ Fast full scan (recommended)

```
nmap -p- --min-rate 5000 192.168.1.10
```

◆ Specific ports

```
nmap -p 22,80,443 192.168.1.10
```

3 UDP Scanning (Often forgotten, high value)

```
nmap -sU --top-ports 100 192.168.1.10
```

(UDP is slow—mention this in interviews)

4 Service & Version Detection

◆ Detect service versions

```
nmap -sV 192.168.1.10
```

◆ Aggressive version detection

```
nmap -sV --version-all 192.168.1.10
```

5 OS Detection

◆ OS fingerprinting

```
sudo nmap -O 192.168.1.10
```

◆ OS detection with guess

```
sudo nmap -O --osscan-guess 192.168.1.10
```

6 Default Scripts (Very Important)

◆ Run default NSE scripts

```
nmap -sC 192.168.1.10
```

7 Aggressive Scan (Interview Favorite)

```
sudo nmap -A 192.168.1.10
```

Includes:

- OS detection
 - Version detection
 - Default scripts
 - Traceroute
-

8 Web Enumeration (HTTP/HTTPS)

◆ Identify web technologies

```
nmap --script http-enum 192.168.1.10
```

◆ Check HTTP methods

```
nmap --script http-methods 192.168.1.10
```

◆ Robots.txt

```
nmap --script http-robots.txt 192.168.1.10
```

9 Vulnerability Enumeration (Safe NSE scripts)

◆ Run common vuln scripts

```
nmap --script vuln 192.168.1.10
```

◆ SMB vulnerabilities

```
nmap --script smb-vuln* -p 445 192.168.1.10
```

10 Authentication & Access Enumeration

◆ SMB shares

```
nmap --script smb-enum-shares -p 445 192.168.1.10
```

◆ SMB users

```
nmap --script smb-enum-users -p 445 192.168.1.10
```

11 Firewall & IDS Evasion (Explain Carefully)

◆ Fragment packets

```
nmap -f 192.168.1.10
```

◆ Decoy scan

```
nmap -D RND:10 192.168.1.10
```

◆ Change source port

```
nmap --source-port 53 192.168.1.10
```

⚠ Say “only in authorized environments”

12 Performance & Timing

◆ Fast scan

```
nmap -T4 192.168.1.10
```

◆ Slow / stealthy scan

```
nmap -T2 192.168.1.10
```

13 Output Formats (Reporting)

◆ Normal output

```
nmap -oN scan.txt 192.168.1.10
```

◆ XML output

```
nmap -oX scan.xml 192.168.1.10
```

◆ All formats

```
nmap -oA fullscan 192.168.1.10
```

14 Real-World Enumeration Flow (Say This)

```
nmap -sn 192.168.1.0/24  
nmap -p- --min-rate 5000 192.168.1.10  
nmap -sC -sV -O -A 192.168.1.10
```



Interview Explanation:

“I start with host discovery, enumerate all ports, then identify services, versions, OS, and finally run scripts for deeper enumeration.”



Final Tip (Interview Gold)

If asked:

“What’s the most important Nmap flag?”

Answer:

“-sV and -sC because knowing *what* is running is more valuable than just knowing *which port is open*.”

If you want next, I can give you:

- **Nmap → Exploitation mapping**
- **Blue-team detection of Nmap scans**
- **NSE scripting basics**
- **Real interview trick questions**

Just say the word.